

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Answer:

Aim:

To implement secure password storage and verification using PBKDF2 with SHA-256, and analyze how salting and key stretching protect passwords against brute-force and rainbow table attacks.

Concept:

PBKDF2 generates a derived key from a password using a random salt and repeated hashing. The salt makes hashes unique, while repeated iterations increase the computational cost of password guessing.

Python Program:

```
import hashlib
```

```
import os
```

```
import time
```

```
# Password hashing using PBKDF2
```

```
def hash_password(password):
```

```
    salt = os.urandom(16)
```

```
    start = time.perf_counter()
```

```
    key = hashlib.pbkdf2_hmac(
```

```
        'sha256',
```

```
        password.encode(),
```

```
        salt,
```

```

        100000
    )

    end = time.perf_counter()

    return salt, key, (end - start) * 1000

# Password verification
def verify_password(password, salt, stored_key):

    key = hashlib.pbkdf2_hmac(

        'sha256',

        password.encode(),

        salt,

        100000

    )

    return key == stored_key

# Main program

password = input("Enter password: ")

salt, stored_key, time_taken = hash_password(password)

```

```

print("\n--- Password Storage ---")

print("Salt:", salt.hex())

print("Stored Hash:", stored_key.hex())

print("Hashing Time: %.2f ms" % time_taken)

# Verify correct password

entered_password = input("\nEnter password for verification: ")

if verify_password(entered_password, salt, stored_key):

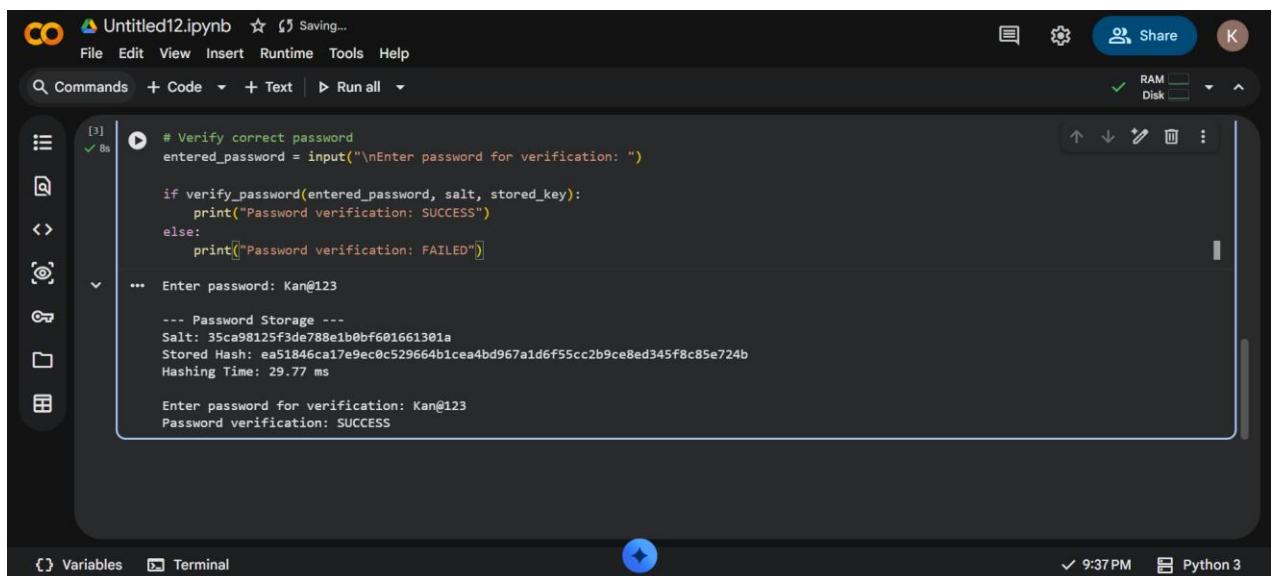
    print("Password verification: SUCCESS")

else:

    print("Password verification: FAILED")

```

OUTPUT:



The screenshot shows a Jupyter Notebook window titled 'Untitled12.ipynb'. The code cell contains the following Python code:

```

# Verify correct password
entered_password = input("\nEnter password for verification: ")

if verify_password(entered_password, salt, stored_key):
    print("Password verification: SUCCESS")
else:
    print("Password verification: FAILED")

```

The output of the code execution is displayed below the code cell:

```

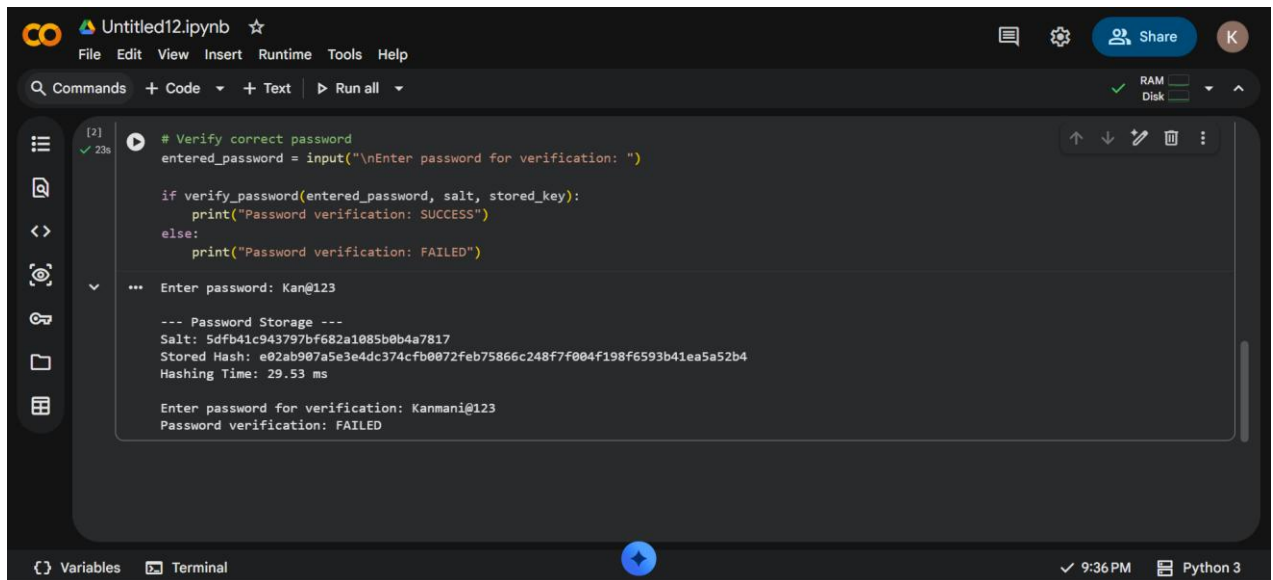
... Enter password: Kan@123

--- Password Storage ---
Salt: 35ca98125f3de788e1b0bf601661301a
Stored Hash: ea51846ca17e9ec0c529664b1cea4bd967a1d6f55cc2b9ce8ed345f8c85e724b
Hashing Time: 29.77 ms

Enter password for verification: Kan@123
Password verification: SUCCESS

```

The interface also shows a 'Variables' tab at the bottom left and a 'Terminal' tab at the bottom center. The status bar at the bottom right indicates the time as 9:37 PM and the Python version as Python 3.



```
[2] ✓ 23s # Verify correct password
entered_password = input("\nEnter password for verification: ")

if verify_password(entered_password, salt, stored_key):
    print("Password verification: SUCCESS")
else:
    print("Password verification: FAILED")

... Enter password: Kan@123

--- Password Storage ---
Salt: 5dfb41c943797bf682a1085b0b4a7817
Stored Hash: e02ab907a5e3e4dc374cfb0072feb75866c248f7f004f198f6593b41ea5a52b4
Hashing Time: 29.53 ms

Enter password for verification: Kanmani@123
Password verification: FAILED
```

HOW SECURITY IS IMPROVED:

1. Salting:

A random salt is added to each password before hashing. Therefore, even if two users have the same password, their hashes will be different. This reduces the effectiveness of rainbow table attacks.

2. Key Stretching:

PBKDF2 performs 100,000 iterations of hashing. This increases the time required for each password guess, making brute-force and dictionary attacks slower.

3. Performance Overhead:

More iterations provide better security but require more processing time. Thus, there is a small performance overhead during password hashing and verification.

ADVANTAGES:

- Protects against rainbow table attacks.
- Slows down brute-force attacks.
- Passwords are not stored directly.
- Random salts produce unique hashes.

CONCLUSION:

PBKDF2 with SHA-256, random salting, and key stretching provides secure password storage and makes password-cracking attacks more difficult.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Aim:

To implement HMAC using SHA-256 for message authentication and analyze how HMAC provides both message integrity and authenticity.

Concept:

HMAC (Hash-based Message Authentication Code) combines a secret key with a cryptographic hash function such as SHA-256. Unlike simple hashing, HMAC requires a secret key, allowing the receiver to verify both the integrity and authenticity of the message.

Python program:

```
import hmac

import hashlib

# Secret key
key = b"secretkey123"

# Message
message = input("Enter message: ").encode()

# Generate HMAC using SHA-256
mac = hmac.new(key, message, hashlib.sha256)

print("\n--- HMAC Generation ---")

print("Message:", message.decode())

print("HMAC:", mac.hexdigest())
```



```
# Verify the message
```

```
received_message = input("\nEnter message for verification: ").encode()
```

```
received_mac = hmac.new(key, received_message, hashlib.sha256)
```

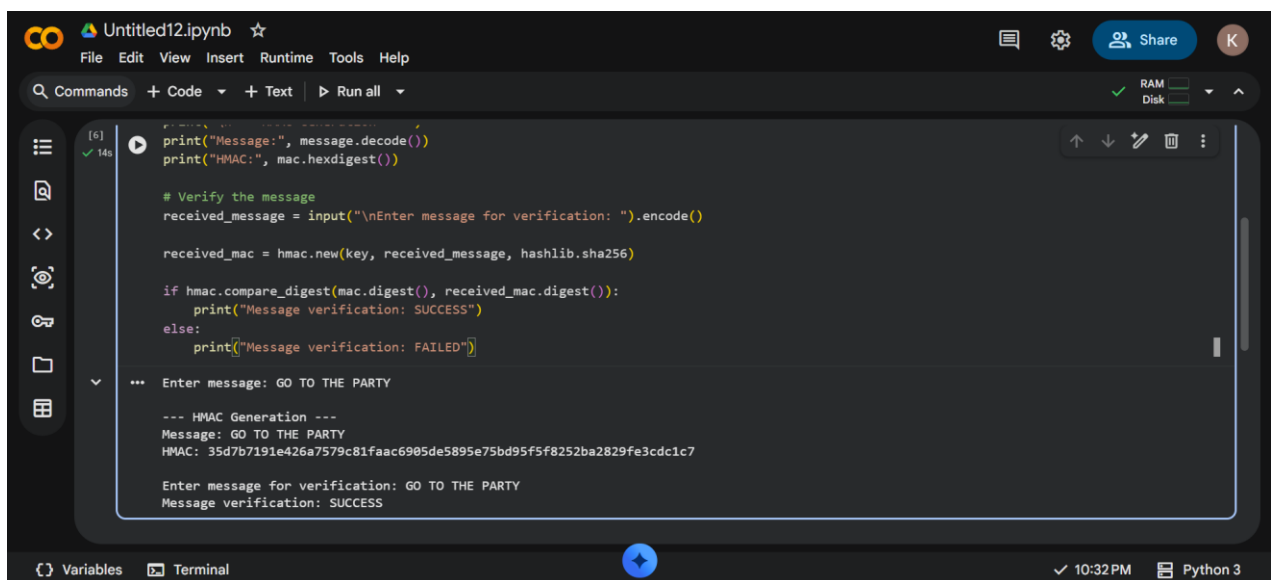
```
if hmac.compare_digest(mac.digest(), received_mac.digest()):
```

```
    print("Message verification: SUCCESS")
```

```
else:
```

```
    print("Message verification: FAILED")
```

OUTPUT:



A screenshot of a Jupyter Notebook interface titled 'Untitled12.ipynb'. The code cell [6] contains the verification logic. The output shows the message 'GO TO THE PARTY' being entered, the HMAC being generated, and the verification succeeding.

```
[6] ✓ 14s
print("Message:", message.decode())
print("HMAC:", mac.hexdigest())

# Verify the message
received_message = input("\nEnter message for verification: ").encode()

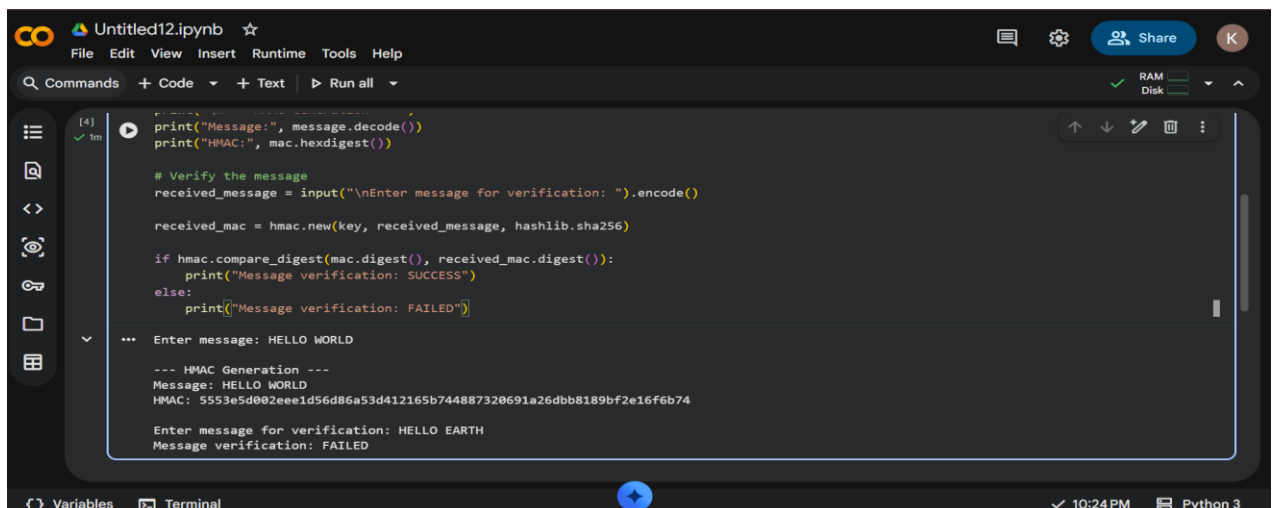
received_mac = hmac.new(key, received_message, hashlib.sha256)

if hmac.compare_digest(mac.digest(), received_mac.digest()):
    print("Message verification: SUCCESS")
else:
    print("Message verification: FAILED")

... Enter message: GO TO THE PARTY

--- HMAC Generation ---
Message: GO TO THE PARTY
HMAC: 35d7b7191e426a7579c81faac6905de5895e75bd95f5f8252ba2829fe3cdc1c7

Enter message for verification: GO TO THE PARTY
Message verification: SUCCESS
```



A screenshot of a Jupyter Notebook interface titled 'Untitled12.ipynb'. The code cell [4] contains the verification logic. The output shows the message 'HELLO WORLD' being entered, the HMAC being generated, and then 'HELLO EARTH' being entered for verification, which fails.

```
[4] ✓ 1m
print("Message:", message.decode())
print("HMAC:", mac.hexdigest())

# Verify the message
received_message = input("\nEnter message for verification: ").encode()

received_mac = hmac.new(key, received_message, hashlib.sha256)

if hmac.compare_digest(mac.digest(), received_mac.digest()):
    print("Message verification: SUCCESS")
else:
    print("Message verification: FAILED")

... Enter message: HELLO WORLD

--- HMAC Generation ---
Message: HELLO WORLD
HMAC: 5553e5d002eee1d56d86a53d412165b744887320691a26dbb8189bf2e16f6b74

Enter message for verification: HELLO EARTH
Message verification: FAILED
```

SECURITY ANALYSIS:

1. Integrity:

HMAC detects whether the message has been modified during transmission. Even a small change in the message produces a different HMAC.

2. Authenticity:

HMAC uses a shared secret key. Only someone possessing the correct key can generate a valid HMAC, helping verify that the message came from an authorized sender.

3. HMAC vs Simple Hashing:

Simple Hash	HMAC
Uses only the message	Uses message + secret key
Provides integrity checking	Provides integrity + authentication
Anyone can calculate the hash	Only someone with the key can create a valid HMAC

ADVANTAGES:

- Ensures message integrity.
- Provides message authentication.
- Uses SHA-256 for strong hashing.
- Detects even small message modifications.

CONCLUSION:

HMAC-SHA256 provides stronger message security than simple hashing because it uses a secret key. It ensures that the message has not been modified and helps verify that it was generated by an authorized party.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

AIM:

To simulate X.509 certificate chain validation and understand how trust is established from a Root CA to an End-Entity certificate.

CONCEPT:

An X.509 certificate contains information about an entity and is digitally signed by a Certificate Authority (CA). A certificate chain establishes trust as:

Root CA → Intermediate CA → End-Entity Certificate

The Root CA is trusted directly. Each certificate is checked to ensure that it is correctly signed, valid, and not revoked.

PYTHON PROGRAM:

Simulated Certificate Chain Validation

```
certificates = {  
    "Root CA": {  
        "issuer": "Root CA",  
        "valid": True,  
        "revoked": False  
    },  
    "Intermediate CA": {  
        "issuer": "Root CA",  
        "valid": True,  
        "revoked": False  
    },  
    "End Entity": {
```

```
    "issuer": "Intermediate CA",  
    "valid": True,  
    "revoked": False  
}  
}
```

```
def validate_certificate(name):  
    cert = certificates[name]  
  
    if not cert["valid"]:  
        return False, "Certificate is invalid"  
  
    if cert["revoked"]:  
        return False, "Certificate is revoked"  
  
    if name != "Root CA":  
        issuer = cert["issuer"]  
  
        if issuer not in certificates:  
            return False, "Issuer not trusted"  
  
        valid, message = validate_certificate(issuer)  
  
        if not valid:  
            return False, message
```

```
return True, "Certificate is valid"
```

```
# Validate certificate chain
```

```
result, message = validate_certificate("End Entity")
```

```
print("Certificate Chain Validation")
```

```
print("-----")
```

```
print("Root CA → Intermediate CA → End Entity")
```

```
if result:
```

```
    print("Status: VALID")
```

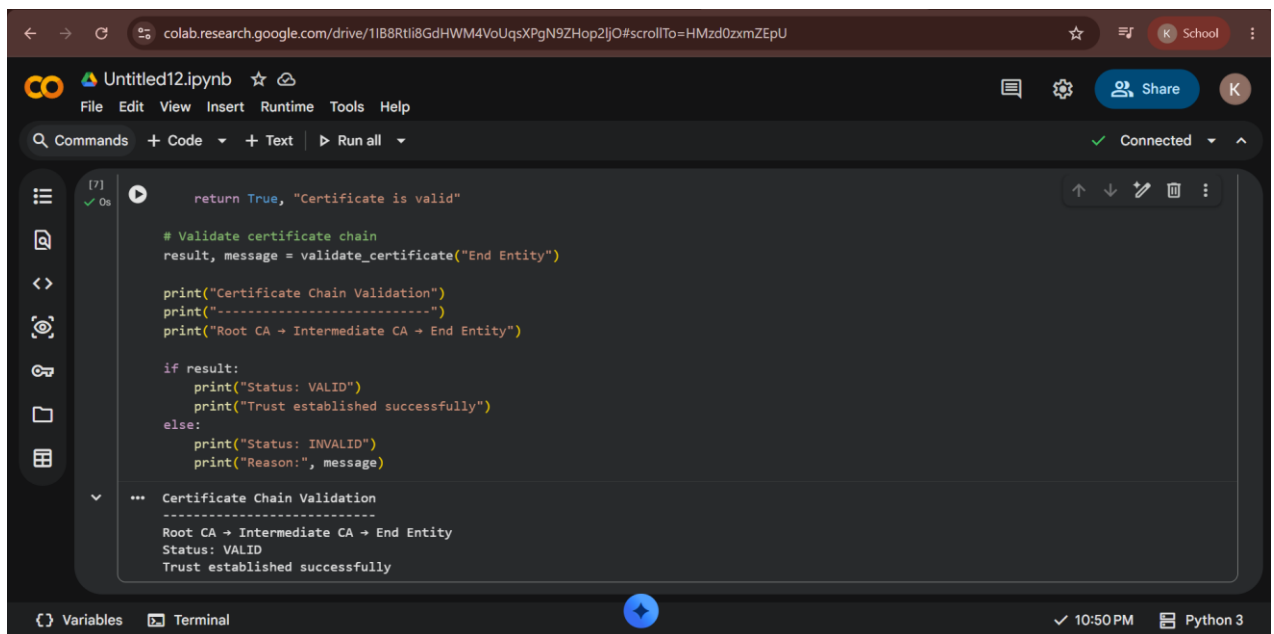
```
    print("Trust established successfully")
```

```
else:
```

```
    print("Status: INVALID")
```

```
    print("Reason:", message)
```

OUTPUT:



```
colab.research.google.com/drive/1IB8Rti8GdHWM4VoUqsXPgN9ZHop2JjO#scrollTo=HMzd0zxmZEPU

Untitled12.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connected

[7]
✓ Os
return True, "Certificate is valid"

# Validate certificate chain
result, message = validate_certificate("End Entity")

print("Certificate Chain Validation")
print("-----")
print("Root CA → Intermediate CA → End Entity")

if result:
    print("Status: VALID")
    print("Trust established successfully")
else:
    print("Status: INVALID")
    print("Reason:", message)

*** Certificate Chain Validation
-----
Root CA → Intermediate CA → End Entity
Status: VALID
Trust established successfully

Variables Terminal
✓ 10:50 PM Python 3
```

SECURITY ANALYSIS:

1. Trust Establishment:

The Root CA is trusted directly. The Intermediate CA is trusted because it is issued by the Root CA, and the End-Entity certificate is trusted through the Intermediate CA.

2. Invalid Certificate:

If a certificate is invalid or its issuer is not trusted, the chain validation fails.

3. Revoked Certificate:

A revoked certificate is rejected even if its signature is otherwise valid.

ADVANTAGES:

- Establishes a trusted identity.
- Detects invalid certificates.
- Prevents trust in unauthorized certificates.
- Helps secure HTTPS and other network communications.

CONCLUSION:

Certificate chain validation establishes trust by verifying certificates from the End Entity back to the trusted Root CA. Invalid, untrusted, or revoked certificates are rejected, helping prevent unauthorized communication.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations

AIM:

To simulate a replay attack in an authentication system and prevent it using a nonce (random number).

CONCEPT:

A replay attack occurs when an attacker captures a valid authentication request and sends the same request again to gain unauthorized access.

A nonce is a randomly generated number used only once. The server checks whether the nonce has already been used. If it has, the request is rejected.

PYTHON PROGRAM:

```
import random

# Server generates a unique nonce
nonce = random.randint(1000, 9999)

print("Server generated nonce:", nonce)

# Client sends authentication request
received_nonce = int(input("Enter nonce for authentication: "))

# First authentication
if received_nonce == nonce:
    print("Authentication SUCCESS")
    used_nonce = received_nonce
else:
```

```
print("Authentication FAILED")
```

```
used_nonce = None
```

```
# Simulate replay attack
```

```
replay_nonce = int(input("\nEnter the same nonce again (Replay Attack): "))
```

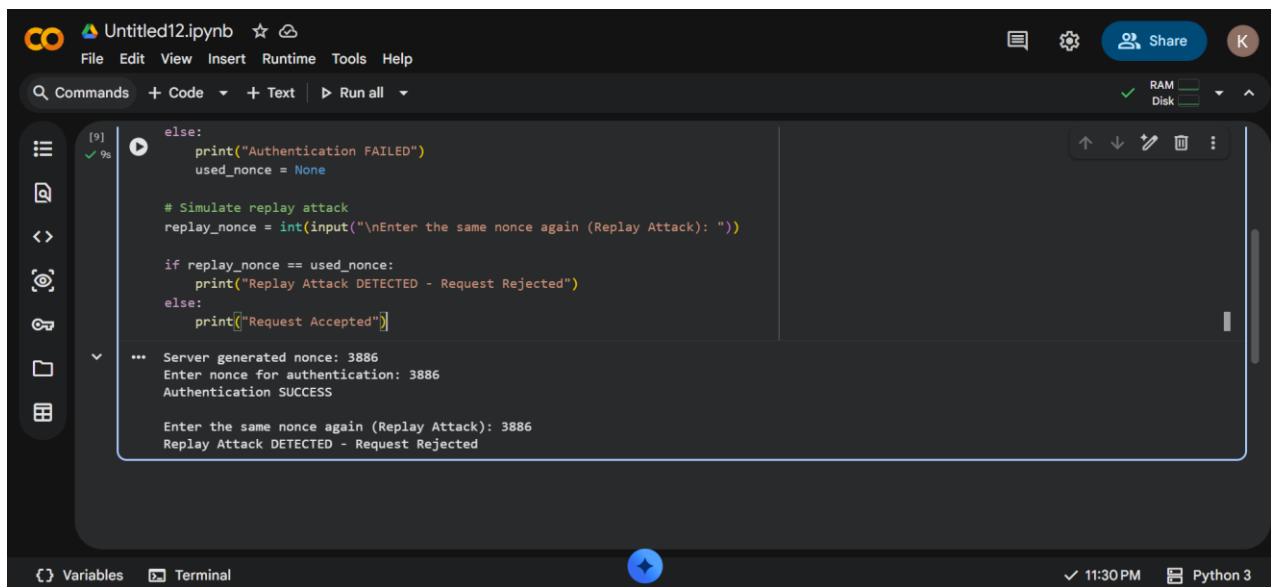
```
if replay_nonce == used_nonce:
```

```
    print("Replay Attack DETECTED - Request Rejected")
```

```
else:
```

```
    print("Request Accepted")
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "Untitled12.ipynb". The code cell contains the following Python code:

```
else:
    print("Authentication FAILED")
    used_nonce = None

# Simulate replay attack
replay_nonce = int(input("\nEnter the same nonce again (Replay Attack): "))

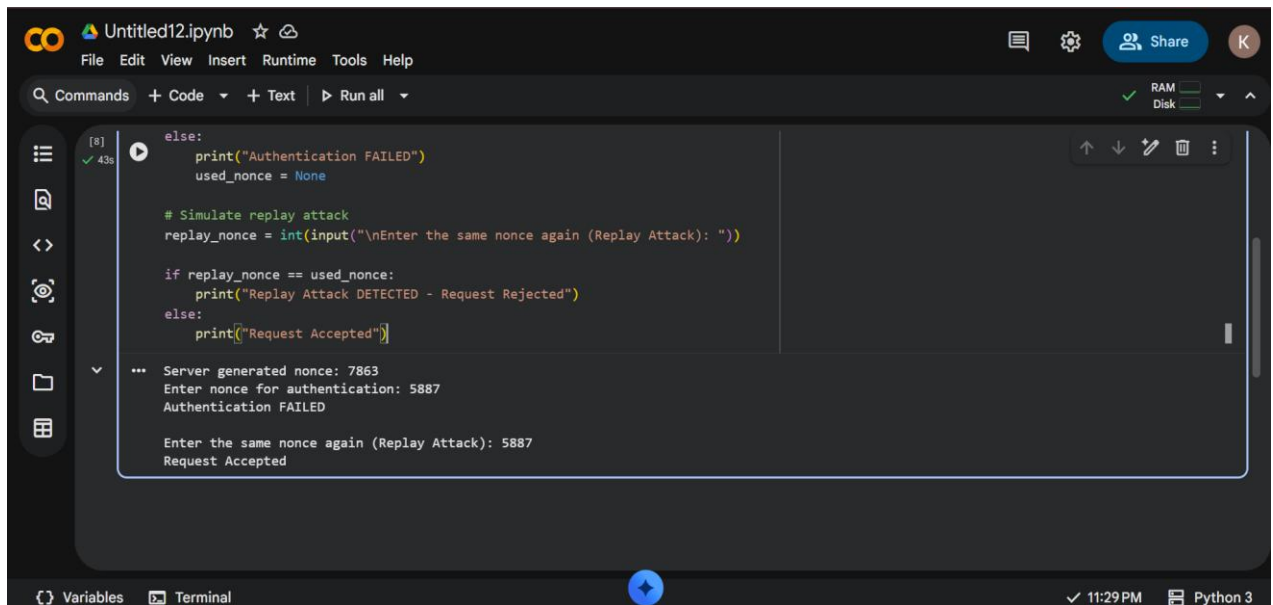
if replay_nonce == used_nonce:
    print("Replay Attack DETECTED - Request Rejected")
else:
    print("Request Accepted")
```

The output of the code is displayed below the code cell:

```
*** Server generated nonce: 3886
Enter nonce for authentication: 3886
Authentication SUCCESS

Enter the same nonce again (Replay Attack): 3886
Replay Attack DETECTED - Request Rejected
```

The interface includes a sidebar on the left with icons for file explorer, search, and other notebook functions. The top bar shows the file name, a star icon, and a share button. The bottom bar displays the status of the notebook, including the time (11:30 PM) and the Python version (Python 3).



```
Untitled12.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM
Disk

[8] 43s
else:
    print("Authentication FAILED")
    used_nonce = None

# Simulate replay attack
replay_nonce = int(input("\nEnter the same nonce again (Replay Attack): "))

if replay_nonce == used_nonce:
    print("Replay Attack DETECTED - Request Rejected")
else:
    print("Request Accepted")

...
Server generated nonce: 7863
Enter nonce for authentication: 5887
Authentication FAILED

Enter the same nonce again (Replay Attack): 5887
Request Accepted

Variables Terminal 11:29 PM Python 3
```

SECURITY ANALYSIS:

1. Replay Attack:

An attacker can capture a valid authentication request and resend it later. Without protection, the server may accept the repeated request.

2. Nonce Prevention:

The server generates a unique nonce for each authentication session. Once the nonce is used, it cannot be accepted again. Therefore, replayed requests are rejected.

3. Security Improvement:

Nonce-based authentication ensures that an old valid request cannot simply be reused by an attacker.

LIMITATIONS:

- The nonce must be unpredictable and unique.
- The server must keep track of used nonces.
- If nonce management is poorly implemented, replay attacks may still occur.
- A nonce alone does not protect the message if the authentication mechanism itself is insecure.

CONCLUSION:

A replay attack reuses a previously captured valid authentication request. Using a unique nonce makes each authentication request different and prevents an old request from being accepted again, thereby improving authentication security.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

AIM

To implement and compare RSA and ECDSA digital signature algorithms based on key generation, signing, and verification time.

CONCEPT

A digital signature is used to verify the authenticity and integrity of a message.

- RSA uses large integer-based public and private keys.
- ECDSA uses elliptic curve cryptography and generally provides comparable security with smaller keys.

The performance is compared using key generation time, signing time, and verification time.

PYTHON PROGRAM:

```
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
from cryptography.hazmat.primitives import hashes
import time
```

```
message = b"Hello, this is a digital signature test."
```

```
# ===== RSA =====
start = time.perf_counter()
```

```
rsa_private = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
rsa_keygen = (time.perf_counter() - start) * 1000
```

```
start = time.perf_counter()
```

```
rsa_signature = rsa_private.sign(
    message,
    padding.PKCS1v15(),
    hashes.SHA256()
)
```

```

rsa_sign = (time.perf_counter() - start) * 1000

start = time.perf_counter()

rsa_private.public_key().verify(
    rsa_signature,
    message,
    padding.PKCS1v15(),
    hashes.SHA256()
)

rsa_verify = (time.perf_counter() - start) * 1000

# ===== ECDSA =====
start = time.perf_counter()

ecdsa_private = ec.generate_private_key(ec.SECP256R1())

ecdsa_keygen = (time.perf_counter() - start) * 1000

start = time.perf_counter()

ecdsa_signature = ecdsa_private.sign(
    message,
    ec.ECDSA(hashes.SHA256())
)

ecdsa_sign = (time.perf_counter() - start) * 1000

start = time.perf_counter()

ecdsa_private.public_key().verify(
    ecdsa_signature,
    message,
    ec.ECDSA(hashes.SHA256())
)

ecdsa_verify = (time.perf_counter() - start) * 1000

# ===== RESULTS =====
print("Digital Signature Performance Comparison")

```

```

print("-----")

print("\nRSA:")
print("Key Generation Time:", round(rsa_keygen, 3), "ms")
print("Signing Time:", round(rsa_sign, 3), "ms")
print("Verification Time:", round(rsa_verify, 3), "ms")

print("\nECDSA:")
print("Key Generation Time:", round(ecdsa_keygen, 3), "ms")
print("Signing Time:", round(ecdsa_sign, 3), "ms")
print("Verification Time:", round(ecdsa_verify, 3), "ms")

```

OUTPUT:

The screenshot shows a Jupyter Notebook interface with the following code and output:

```

[11] print("\nRSA:")
      print("Key Generation Time:", round(rsa_keygen, 3), "ms")
      print("Signing Time:", round(rsa_sign, 3), "ms")
      print("Verification Time:", round(rsa_verify, 3), "ms")

      print("\nECDSA:")
      print("Key Generation Time:", round(ecdsa_keygen, 3), "ms")
      print("Signing Time:", round(ecdsa_sign, 3), "ms")
      print("Verification Time:", round(ecdsa_verify, 3), "ms")

```

The output, titled "Digital Signature Performance Comparison", shows the following results:

```

-----

RSA:
Key Generation Time: 19.475 ms
Signing Time: 1.777 ms
Verification Time: 0.131 ms

ECDSA:
Key Generation Time: 0.331 ms
Signing Time: 5.341 ms
Verification Time: 0.186 ms

```

PERFORMANCE ANALYSIS

Parameter	RSA	ECDSA
Key Generation	Higher	Lower
Signing	Moderate	Fast
Verification	Very fast	Fast
Key Size	Larger	Smaller

Actual timings may vary each time the program runs.

SECURITY ANALYSIS:

RSA:

Widely used and provides fast signature verification. It requires larger keys for strong security.

ECDSA:

Provides strong security with much smaller keys, making it suitable for mobile devices, IoT devices, and systems with limited storage or bandwidth.

ADVANTAGESE:

- Digital signatures provide message integrity and authenticity.
- RSA is widely supported and easy to deploy.
- ECDSA provides strong security with smaller keys.
- Performance can be measured objectively using execution time.

CONCLUSION

Both RSA and ECDSA provide secure digital signatures. RSA is suitable for applications requiring broad compatibility and fast verification, while ECDSA is preferred where smaller keys, lower storage, and efficient communication are important.